# Session 1: Python Fundamentals

Trainer Reference Guide

## Purpose and outcomes

Learners create variables, inspect types, convert input, use arithmetic/comparison/logical operators, and produce formatted output with f-strings.

## Examples and exercises

- 01_variables_and_types.ipynb models an employee with string, integer, float, Boolean, and None values. Guided work creates department and employee-count variables. The independent challenge calculates service years.
- 02_input_output.py captures name, email, department, and age, normalizes text, and returns a registration summary. Highlight that input always returns text.
- 03_operators.py calculates gross pay, eligibility, and a service milestone.
- exercise_01_profile.py is guided variable/f-string practice. Success: one sentence uses all fields with correct types.
- exercise_02_calculator.py independently captures two decimals and an operator. Discuss division by zero as a preview, not a requirement yet.
- challenge_employee_badge.py combines input, normalization, alignment, and formatting into Project 1.

## Trainer key points specific to this module

Contrast expression, statement, variable, and value with concrete output. Show ‘type()‘ and conversions live. Watch for quoted numbers, invalid identifiers, and string concatenation type errors. Ask which portal fields are naturally Boolean or optional.

## Assessment evidence

Learners can explain why ‘int(input())‘ is needed and produce a readable badge without copying the solution.

## Delivery pattern

1. Explain the idea briefly and ask learners to predict a result.
2. Run the simple example, then connect it to an employee/user workflow.
3. Pair learners for the guided task; require an individual attempt next.
4. Use the challenge as the cumulative project increment.
5. Review the quiz and ask learners to explain, not only show, their code.

## Trainer key points

- Keep live coding visible and deliberate. Normalize reading tracebacks.
- Ask what type and value each expression produces before running it.
- Let TODO files remain student-owned; open solutions only during review.
- Check boundary and failure cases, not only the happy path.
- End by tracing how the topic appears in the final portal.